AutoDSC: An Automated Data Stream Clustering Framework

Anonymous

***Abstract*—We introduce AutoDSC (Automated Data Stream Clustering), an AutoML framework that selects a clustering algorithm and tunes its hyperparameters online, in a single pass over a drifting data stream and without access to labels. This matters because streams in IoT, network-monitoring, and sensor applications drift continuously, so no fixed algorithm or configuration stays optimal, yet clustering lacks a universal objective to optimize, which is why AutoML has transformed supervised learning but barely reached unsupervised streams. AutoDSC casts the task as an online combined algorithm-selection and hyperparameter-optimization (CASH) problem and solves it with Population-Guided Stochastic Search Optimization (PGSSO) and a label-free multi-objective cluster-validity criterion that adapts as the distribution shifts. We evaluate AutoDSC across ten real-world and synthetic streams against state-of-the-art stream-clustering and AutoML baselines. AutoDSC attains the best mean rank among all compared methods on both the Adjusted Rand Index and CMM and the highest mean ARI overall, while remaining the most efficient of the AutoML stream clusterers at high throughput and low memory.**

***Index Terms*—data stream clustering, AutoML, concept drift, hyperparameter optimization, online learning**

I. Introduction

On a data stream the data-generating distribution drifts over time, so no single clustering algorithm or hyperparameter setting stays optimal from start to finish; the right choice must itself change as the stream evolves. Automated machine learning (AutoML) was built to make exactly this kind of algorithm and hyperparameter choice automatically [1], but almost entirely for batch supervised learning: clustering has no universally applicable objective to optimize [2], and streaming adds single-pass, bounded-memory, and concept-drift constraints that batch AutoML cannot meet [3]. We introduce AutoDSC (Automated Data Stream Clustering), an AutoML framework that selects the clustering algorithm and tunes its hyperparameters online, in a single pass over IoT, network, and sensor streams [4], and adapts continuously as the distribution drifts, all without labels (Figure 1).

Existing online AutoML frameworks [5]–[8] target supervised tasks, leaving unsupervised streaming clustering underserved. At present, ConfStream [9] and OnlineAutoClust [10] are the only established AutoML (CASH-style) solutions for stream clustering, and the recent MOStream [11] self-optimizes from a rule-based angle that reconfigures structural components rather than searching the joint algorithm–hyperparameter space. These solutions are limited: ConfStream relies on a single clustering metric, while OnlineAutoClust uses computationally intensive offline search, both ill-suited to real-time drifting streams [12]. AutoDSC instead formalizes the streaming CASH problem and solves it with a single-pass guided search and a label-free multi-objective criterion, adapting to drift through continual per-window re-evaluation. As evidence we report ranks and quality across ten streams and an efficiency comparison; we do not claim a universally best clusterer, only the best automated selection among the algorithms in our portfolio. We evaluate AutoDSC against five baselines and make the following key contributions.

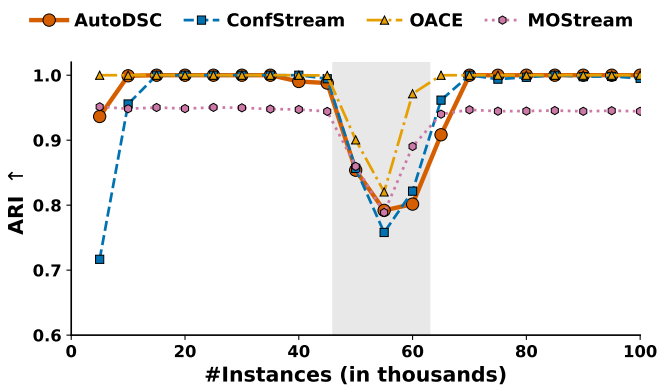
- We cast online stream clustering as a streaming combined algorithm selection and hyperparameter optimization (CASH) problem that jointly chooses the clustering algorithm and tunes its hyperparameters while the data distribution drifts, and we present AutoDSC, a single-pass AutoML framework that solves it autonomously without manual tuning or labels.
- To propose new configurations, AutoDSC introduces Population-Guided Stochastic Search Optimization (PGSSO), which maintains a recency-weighted elite archive and an adaptively controlled exploration rate so that promising algorithm and hyperparameter settings are found quickly and the population recovers implicitly as the stream drifts.
- To choose among configurations without ground-truth labels, AutoDSC scores each one with a normalized multi-objective criterion that combines three complementary internal cluster-validity indices, yielding robust selection where the single-metric criteria of prior systems fail.
- Across ten real and synthetic streams, we show that AutoDSC attains the best mean rank on both the Adjusted Rand Index (ARI) [13] and CMM [14] and the highest mean ARI among all methods while being the most efficient of the AutoML stream clusterers.

II. Background and Related Work

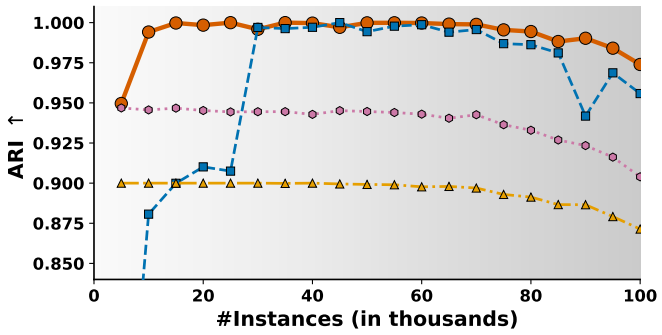
We first review the data stream clustering setting, its algorithms, and how clustering quality is measured online, and then survey automated machine learning and its existing realizations for stream clustering, which together motivate AutoDSC.

A. Data Stream Clustering

A data stream is an unbounded, high-velocity sequence of instances that a learner must process online rather than as a stored dataset. Following Bifet et al. [3], a stream learner should satisfy five requirements: (**R1**) process one instance at a time and inspect it at most once; (**R2**) use a bounded amount



(a) *Merge-Split*: a discrete drift event (shaded)



(b) *Expanding*: continuous drift (left-to-right gradient)

Fig. 1. AutoDSC adapts to concept drift without labels. Per-window ARI (seed-averaged) on two synthetic streams: under a discrete merge-then-split event (a, shaded) and under continuously growing within-cluster variance (b, gradient). AutoDSC (thick red line) sustains the highest clustering quality and recovers fastest, while the fixed and competing adaptive baselines degrade or plateau below it.

of time per instance; (**R3**) use a bounded amount of memory; (**R4**) be ready to produce an answer, such as a prediction, clustering, or set of patterns, at any time; and (**R5**) adapt to temporal changes in the data. These constraints rule out the multi-pass, full-data access that batch clustering and offline AutoML assume. Because the data-generating process evolves, streams also exhibit concept drift, meaning the distribution of the data, and for clustering the number, shape, and location of the clusters, changes over time, whether abruptly, gradually, or incrementally [3], [15]. A method that fixes its model or configuration in advance therefore degrades as the stream moves away from the regime it was tuned for, which is exactly what makes requirement R5 hard.

Classical stream clustering algorithms meet these requirements and fall into a few families distinguished by how they summarize the stream. The line begins with BIRCH [16], whose clustering-feature tree first showed that a single-scan, bounded-memory summary could cluster very large datasets and seeded the micro-cluster summaries used by later stream methods. Micro-cluster summarization, pioneered by CluStream [17], maintains pyramidal time-frame statistics for scalable analysis; density-based methods such as DenStream [18] and DBSTREAM [19] discover arbitrarily shaped clusters through core, potential, and outlier micro-clusters and a shared-density graph; and k-means-based methods maintain centroids online, where mini-batch optimization enables web-scale clustering at low latency [20] and StreamKMeans [21] replaces the k-medians step of stream local search [22] with incremental k-means. These algorithms are effective once configured, but each fixes its algorithm and hyperparameters in advance, which motivates automating their selection. Quality is measured online with the interleaved test-then-train, or prequential, protocol [3], in which each instance is used to test the model before it updates on it. As labels are unavailable during the stream, configuration choices rely on internal cluster validity indices (CVIs) such as the Silhouette, Davies–Bouldin, and Calinski–Harabasz scores, and since no single CVI is reliable across cluster shapes, densities, and counts, combining complementary indices is more robust than any one alone [23], [24]; for label-based scoring, Kremer et al. [14] introduced the Cluster Mapping Measure (CMM), which unlike classical external indices accounts for emerging, splitting, and moving clusters, so we evaluate with CMM alongside the chance-corrected Adjusted Rand Index (Section V).

B. Automated Machine Learning

Automated Machine Learning removes the manual burden of algorithm selection and hyperparameter optimization, formalized as the combined algorithm selection and hyperparameter optimization (CASH) problem [1]. AutoML is mature for batch supervised learning, and the few streaming AutoML systems likewise target supervised tasks [5], [6]. For clustering, several batch AutoML systems automate the pipeline, such as AutoML4Clust [25], AutoCluster [26], cSmartML [27], ML2DAC [28], TPE-AutoClust [29], and PoAC [30], through meta-learning and pipeline composition, but all assume repeated full-data access and therefore cannot operate under the single-pass, bounded-memory, drifting conditions of a stream.

For stream clustering specifically, ConfStream [9] is an early attempt at automated configuration, using an ensemble of clustering algorithms and configurations that adapt by evaluating quality and sampling nearby configurations proportionally to their performance, but it relies on a single Silhouette Width metric and a limited search. OnlineAutoClust [10] combines Bayesian optimization with clustering ensembles via Tree-structured Parzen Estimators, yet depends on offline tuning, is computationally heavy, and is sensitive to its time budget, which can yield suboptimal pipelines during rapid drift. From a different, rule-based angle, MOStream [11] self-optimizes by reconfiguring four structural design dimensions (the summarizing data structure, the window model, the outlier-detection mechanism, and the refinement strategy) at runtime rather than searching a joint algorithm–hyperparameter space; it is the automated successor of the modular Benne algorithm distilled from the SESAME empirical study of these design aspects [31], which in turn builds on classical summaries such as BIRCH [16]. ConfStream, OnlineAutoClust, and MOStream can thus be read as automating, in different ways, the configuration of these classical stream clusterers, which is

the role AutoDSC fills for the joint algorithm–hyperparameter space. In contrast to all of these approaches, AutoDSC adapts to drift implicitly through continual per-window re-evaluation and an adaptive exploration rate, avoiding both single-metric evaluation and expensive offline optimization.

III. Problem Formulation

A data stream is an unbounded sequence $\mathcal{S} = \{x_1, x_2, \ldots, x_t, \ldots\}$ in which each instance $x_t \in \mathbb{R}^d$ arrives one at a time and must be processed in a single pass under bounded memory. We consume the stream in consecutive windows of fixed size w, writing W for the window currently being processed, and the data-generating distribution may change from one window to the next (concept drift), so the best way to cluster the data is itself time-varying. Given a window W, a clustering algorithm groups its points by similarity without access to labels. Centroid-based methods, for instance, summarize each group by a representative and seek a set C of centroids that minimizes the average distance to the nearest centroid,

$$\mathrm{cost}(C, W) = \frac{1}{|W|} \sum_{x \in W} \min_{c \in C} d(x, c), \tag{1}$$

where d is a distance function, while density- and micro-cluster-based methods optimize related criteria. No single algorithm or objective suits all data, so the appropriate choice varies across datasets and, under drift, across time. Because ground-truth labels are unavailable online, the quality of a clustering produced on W must be judged by an internal cluster-validity score $q(\cdot) \in [0, 1]$ (higher is better) computed from the data and the assignments alone, whereas external indices such as ARI and CMM use labels and are reserved for offline evaluation (Section V).

Let $\mathcal{A}$ be a portfolio of stream-clustering algorithms and $\Theta = \bigcup_{a \in \mathcal{A}} (\{a\} \times \Phi_a)$ the joint space of configurations $\theta = (a, \phi)$ that pair an algorithm a with hyperparameters $\phi \in \Phi_a$. Running θ on the current window yields a clustering with label-free quality $q(\theta; W)$. The combined algorithm-selection-and-hyperparameter-optimization (CASH) problem for a stream is to track, at every window, the configuration

$$\theta^\star = \arg\max_{\theta \in \Theta} \; q(\theta; W), \tag{2}$$

using only the data seen so far and bounded memory. As the distribution drifts, $\theta^\star$ changes over time, so a solver must continually re-evaluate and re-select configurations rather than commit to one. AutoDSC approximates this online optimum with a population-guided search and a label-free multi-objective instantiation of q, described next.

IV. Automated Data Stream Clustering

We introduce Automated Data Stream Clustering (AutoDSC), a novel AutoML framework tailored for non-stationary data stream clustering. AutoDSC processes data in fixed-size windows. At the end of each window, AutoDSC evaluates clustering configurations using multiple normalized validation indices. The framework maintains a recency-weighted elite archive of high-performing configurations to guide future parameter exploration via PGSSO. It adaptively balances exploration and exploitation through dynamic rate adjustment based on performance feedback. Adaptation to distributional change is implicit, since each window the population is re-evaluated and the incumbent re-selected, and a drop in clustering quality automatically raises the exploration rate so new configurations are sought for the new regime. Figure 2 illustrates the high-level workflow of AutoDSC, while Algorithm 1 details the complete operational steps.

To solve the online CASH problem of Section III, AutoDSC maintains a configuration population $\mathcal{A}_t \subseteq \Theta$ of clustering algorithms with different hyperparameter settings, fills the current window W_t as instances arrive, and serves real-time predictions from the best-performing (incumbent) configuration. As each new data point arrives, the system performs incremental updates across all configurations:

$$\forall \theta_i \in \mathcal{A}_t : \quad M_i^{(t)} = \text{LearnOne}(M_i^{(t-1)}, x_t) \tag{3}$$

where $M_i^{(t)}$ represents the model state of configuration θ_i at time t. When the window reaches capacity, the framework initiates evaluation and optimization operations.

When the window fills, AutoDSC evaluates clustering quality using three normalized clustering validation indices, namely the Normalized Silhouette Coefficient NSC $= (1 +$ SC$)/2$, the Normalized Davies-Bouldin Index NDBI $= 1 -$ tanh(DBI), and the Normalized Calinski-Harabasz Index NCHI $=$ tanh(log(1 + CHI)/10), ensuring all metrics are bounded to $[0, 1]$ where higher values indicate better clustering quality. Each configuration is scored on the entire window (the per-point cluster assignments produced during LearnOne are retained for this purpose), so no sub-sampling is required. The expected clustering quality for configuration θ at time t is:

$$\mu_t(\theta) = \frac{1}{3}[\text{NSC} + \text{NDBI} + \text{NCHI}] \tag{4}$$

AutoDSC dynamically adjusts exploration behavior based on performance improvement trends. The exploration rate α_t adapts based on relative improvement:

$$\Delta\mu_t = \mu_t(\theta_t^*) - \mu_{t-1}(\theta_{t-1}^*) \tag{5}$$

$$\alpha_{t+1} = \begin{cases} \max(0.1,\ \alpha_t \cdot (1 - \beta_t)) & \text{if } \Delta\mu_t > 0 \\ \min(1.0,\ \alpha_t \cdot (1 + \beta_t)) & \text{if } \Delta\mu_t \le 0 \end{cases} \qquad \beta_t = \min(\gamma, |r_t|) \tag{6}$$

where $r_t = \Delta\mu_t / |\mu_{t-1}|$ is the relative improvement and β_t is the resulting step, capped at $\gamma = 0.5$ for stability. This parameter-free mechanism decreases exploration when performance improves (favoring exploitation) and increases it when performance stagnates or drops, as happens under concept drift, encouraging broader search to recover.

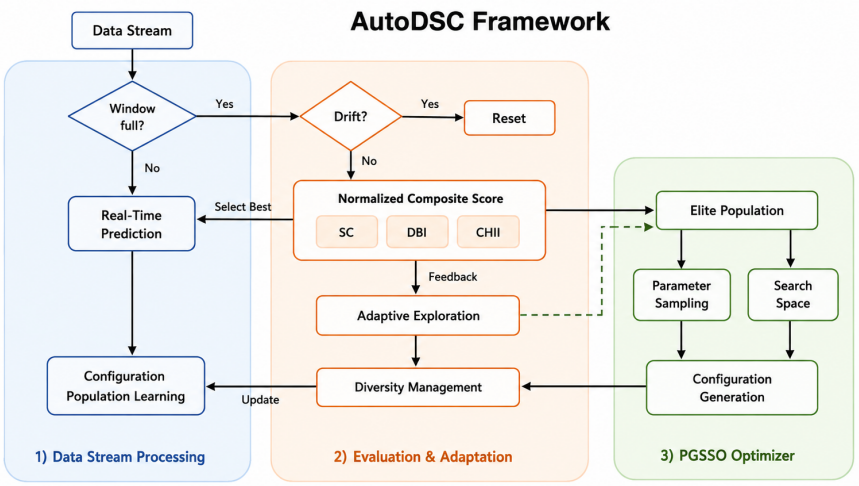


Fig. 2. Overview of AutoDSC. Each window the population of clustering configurations is evaluated by the multi-objective criterion, the incumbent is re-selected, and PGSSO proposes new configurations from the recency-weighted elite archive under an adaptive exploration rate.

The framework employs **Population-Guided Stochastic Search Optimization (PGSSO)** (Algorithm 2) to jointly optimize algorithm selection and hyperparameter tuning. PGSSO maintains an elite archive $\mathcal{E}_t = \{(\theta_i, s_i)\}_{i=1}^{e}$ where each configuration $\theta_i = (a_i, \phi_i)$ pairs an algorithm choice with its hyperparameters and $s_i = \mu_t(\theta_i)$ is its score. To remain responsive to drift, elites are ranked by a *recency-weighted* score $s_i/(1 + g - g_i)$, where g is the current generation and g_i the generation at which θ_i was last scored; stale high-scoring configurations are thus down-weighted in favor of recently effective ones. By tracking which algorithm-hyperparameter combinations perform best, PGSSO naturally learns both which algorithms suit current data characteristics and their optimal settings. The elite population encodes this joint preference, where frequently appearing algorithms indicate superior algorithmic choices, while their parameter distributions reveal effective settings. PGSSO adaptively balances exploration and exploitation through dynamic rate α_t, computing statistical distributions over both algorithm selections (via frequencies) and their parameters (via means μ_j and variances σ_j^2), which together guide sampling of new promising algorithm-hyperparameter configurations.

PGSSO generates new configurations through probabilistic sampling:

$$\theta_{\text{new}} = \begin{cases} \text{Elite-Guided}(\mathcal{E}_t) & \text{with probability } (1 - \alpha_t) \\ \text{Random}(\Theta) & \text{with probability } \alpha_t \end{cases} \tag{7}$$

Elite-guided sampling first selects an algorithm based on elite frequencies, then samples its hyperparameters from learned distributions. Algorithm selection uses:

$$P(a = a_k) = \frac{\sum_{(\theta,s)\in\mathcal{E}_t} \mathbf{I}[a = a_k]}{|\mathcal{E}_t|} \tag{8}$$

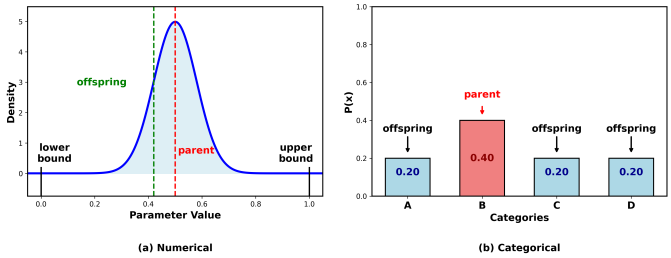


Fig. 3. Parameter Sampling Visualization.

where $a_k \in \mathcal{A}$ represents available algorithms. Given selected algorithm a_k, numerical hyperparameters use Gaussian distributions from elite configurations of that algorithm:

$$\phi_j \sim \mathcal{N}(\mu_j, \sigma_j^2) \text{ where } \mu_j = \frac{1}{|\mathcal{E}_t^{a_k}|} \sum_{(\theta,s)\in\mathcal{E}_t^{a_k}} \phi_j \tag{9}$$

and $\mathcal{E}_t^{a_k} = \{(\theta, s) \in \mathcal{E}_t : a = a_k\}$ contains elite configurations using algorithm a_k. Categorical hyperparameters similarly use frequency-based sampling within algorithm-specific elites. Figure 3 illustrates this hierarchical sampling process.

Population diversity is maintained by computing configuration distances:

$$d(\theta_1, \theta_2) = \begin{cases} 1.0 & \text{if algorithms differ} \\ \frac{1}{|\mathcal{P}|} \sum_{p\in\mathcal{P}} d_p(\theta_1[p], \theta_2[p]) & \text{otherwise} \end{cases} \tag{10}$$

where $\mathcal{P}$ represents common parameters and d_p computes normalized parameter-specific distances. When the population exceeds size K, it is pruned by first retaining the incumbent and top-scoring configurations and then greedily adding the

remaining configuration that is *farthest* (by d) from those already kept, a farthest-first selection that preserves diversity without requiring a fixed acceptance threshold.

Throughout the process, AutoDSC provides real-time clustering predictions using the incumbent (best) configuration:

$$\theta_t^* = \arg\max_{\theta_i \in \mathcal{A}_t} \mu_t(\theta_i) \tag{11}$$

All remaining constants are held fixed across every experiment. The exploration rate is clipped to $[0.1, 1.0]$ with a per-step cap $\gamma = 0.5$, the elite archive holds $e_{\max} = \lfloor K/2 \rfloor$ configurations, and each window PGSSO proposes $n_{\text{gen}} = \max(3, \lfloor K/4 \rfloor)$ new candidates (three at the default $K = 10$) that replace the lowest-ranked members so the population size stays at K. The Calinski–Harabasz score is rescaled by a fixed factor of 10 to bound NCHI to $[0, 1]$. The sensitivity study (Section VI-B) varies only K and w and holds all other constants at these defaults.

A. Computational Complexity

The framework's computational complexity scales efficiently with population size and window parameters. Per-instance processing requires $O(K \cdot C)$ operations where $K = |\mathcal{A}_t|$ and C denotes average algorithm complexity. Window-end evaluation involves $O(K \cdot w \cdot d)$ operations to score each configuration on the full window. Total per-window complexity is $O(K \cdot w \cdot d + K^2)$, where K^2 accounts for diversity-based population management. Crucially, K is fixed, so per-point cost does not grow with stream length. Space complexity is $O((K + e_{\max}) \cdot |\Theta| + w \cdot d)$.

V. Experimental Setup

Our experimental design follows rigorous evaluation protocols to ensure reproducible and statistically valid results.[1]

A. Datasets

We evaluate on 10 datasets comprising 5 real-world and 5 synthetic streams with varying sizes, dimensionalities, and cluster counts, as summarized in Table I. Datasets were selected to (i) cover the real-world streams most frequently used in prior stream-clustering work, spanning a wide range of sizes (11k–101k instances), dimensionalities (9–100 attributes), and cluster counts (3–12), and (ii) probe specific drift mechanisms in isolation through controlled synthetic generators.

The five real-world streams are public benchmarks with natural class structure and distribution change.

- **Insects** [32]: optical-sensor wing-beat features of flying insects, where the species labels and feature distribution shift with ambient temperature over the recording.
- **KDD99** [33]: network-connection records from the KDD Cup 1999 intrusion-detection task, labelled as normal traffic or one of several attack types.
- **PenDigits** [33]: pen-trajectory features of handwritten digits, with one cluster per digit 0–9.

[1]Datasets and code: https://anonymous.4open.science/r/AutoDSC

Algorithm 1: AutoDSC

Input: Stream $\mathcal{S} = \{x_1, x_2, \ldots\}$, $x_t \in \mathbb{R}^d$; window size w; population size K
Output: Cluster predictions $\{y_t\}_{t=1}^{\infty}$ using θ_t^*

$e_{\max} \leftarrow \lfloor K/2 \rfloor$;
$\mathcal{E}_0 \leftarrow \emptyset$, $\alpha_0 \leftarrow 0.5$;
$\mathcal{A}_0 \leftarrow \{\theta_i\}_{i=1}^{K}$ where $\theta_i \sim \text{Uniform}(\Theta)$;
$W_0 \leftarrow \emptyset$, $\theta_0^* \leftarrow \emptyset$;
$\mu_{-1}(\theta_{-1}^*) \leftarrow -\infty$;
foreach $x_t \in \mathcal{S}$ **do**
  $W_t \leftarrow W_{t-1} \cup \{x_t\}$;
  foreach $\theta_i \in \mathcal{A}_t$ **do**
    $\ell_{i,t} \leftarrow \textsc{PredictOne}(M_i, x_t)$; `// store assignment`
    $M_i^{(t)} \leftarrow \textsc{LearnOne}(M_i^{(t-1)}, x_t)$;
  if $|W_t| = w$ **then**
    foreach $\theta_i \in \mathcal{A}_t$ **do**
      $L_i \leftarrow \{\ell_{i,\tau} : x_\tau \in W_t\}$;
      $\mu_t(\theta_i) \leftarrow \textsc{NormCVI}(W_t, L_i)$;
    $\mu_{\text{best}}^{(t)} \leftarrow \max_i \mu_t(\theta_i)$;
    $\alpha_{t+1} \leftarrow \textsc{Adapt}(\alpha_t, \mu_{t-1}(\theta_{t-1}^*), \mu_{\text{best}}^{(t)})$;
    $\mathcal{C}_{\text{new}} \leftarrow \text{PGSSO}(\{(\theta_i, \mu_t(\theta_i))\}, \alpha_{t+1}, \Theta)$;
    $\mathcal{A}_{t+1} \leftarrow \textsc{SelectDiverse}(\mathcal{A}_t \cup \mathcal{C}_{\text{new}}, K)$;
    $\theta_t^* \leftarrow \arg\max_{\theta_i \in \mathcal{A}_{t+1}} \mu_t(\theta_i)$;
    $W_t \leftarrow \emptyset$;

Algorithm 2: PGSSO

Input: Current configurations with scores $\{(\theta_i, s_i)\}$; exploration rate $\alpha \in [0, 1]$; parameter space Θ
Output: New configuration set $\mathcal{C}_{\text{new}}$

$\mathcal{E} \leftarrow \textsc{UpdateElitePopulation}(\{(\theta_i, s_i)\}, e_{\max})$;
$\mathcal{E} \leftarrow \textsc{UpdateParameterDistributions}(\mathcal{E})$;
$\mathcal{C}_{\text{new}} \leftarrow \emptyset$;
for $j \leftarrow 1$ **to** n_{gen} **do**
  $u \leftarrow \text{Uniform}(0, 1)$;
  if $u > \alpha$ **then**
    $\theta_{\text{new}} \leftarrow \textsc{SampleFromElite}(\mathcal{E}, \Theta)$;
  else
    $\theta_{\text{new}} \leftarrow \textsc{RandomSample}(\Theta)$;
  $\mathcal{C}_{\text{new}} \leftarrow \mathcal{C}_{\text{new}} \cup \{\theta_{\text{new}}\}$;
return $\mathcal{C}_{\text{new}}$;

- **Shuttle** [33]: NASA Space Shuttle radiator sensor readings (Statlog), with highly imbalanced operational-state classes.
- **Vehicle** [34]: high-dimensional acoustic and seismic features from a distributed wireless sensor network, labelled by vehicle type.

The five synthetic streams are produced by our reproducible generators (released with the code) and let us control the drift

TABLE I
SUMMARY OF DATASETS

Dataset Name	Type	Instances	Attributes	Clusters
Insects	Real	52,848	33	6
KDD99	Real	100,655	38	12
PenDigits	Real	10,992	16	10
Shuttle	Real	58,000	9	7
Vehicle	Real	98,528	100	3
Expanding	Synth	100,000	5	5
Gradual	Synth	100,000	5	5
Merge-Split	Synth	100,000	5	5
Sudden	Synth	100,000	5	5
Fast-RBF	Synth	100,000	5	5

mechanism while holding dimensionality and cluster count fixed. Four are five-cluster Gaussian-blob streams in $[0, 1]^5$ that differ only in the injected drift, and one is a MOA-style RandomRBF stream [35].

- **Expanding**: cluster centroids stay fixed while the within-cluster variance grows steadily over the stream.
- **Gradual**: centroids translate smoothly at constant speed along fixed random directions.
- **Merge-Split**: clusters converge toward the global mean at mid-stream and then re-separate.
- **Sudden**: centroids jump abruptly at three change points, holding stationary between them.
- **Fast-RBF**: a RandomRBF stream whose class centroids drift rapidly across the feature space.

B. Evaluation Protocol

All experiments use the prequential protocol (Section II) with 10 independent runs using random seeds 1-10 to account for stochastic variability. Clustering quality is measured using the Adjusted Rand Index (ARI) and CMM [14], where ground truth labels are used solely for evaluation, not training or model selection. ARI is the chance-corrected form [13] of the Rand index [36] that counts agreements between the predicted clustering and the ground-truth partition over all pairs of points and adjusts the count for the agreement expected at random, yielding 1 for a perfect match, 0 for random labelling, and negative values for worse-than-random partitions. We also record throughput (instances/s) and peak memory usage (MB) to assess computational efficiency. Statistical significance is assessed using the Friedman test followed by the Nemenyi post-hoc test [37] with $\alpha = 0.05$.

C. Baseline Methods

We compare AutoDSC against five baselines spanning three categories. (i) *AutoML stream-clustering methods*: *ConfStream* [9] and *OnlineAutoClust* (OACE) [10], established frameworks that optimize hyperparameters online. (ii) The *rule-based adaptive* method *MOStream* [11]. (iii) Two *baselines*: *RandomSearch* [38], which shares AutoDSC's framework structure but generates configurations by purely random sampling, a direct comparison to our guided optimization, and the *Default(Best)* baseline of OnlineAutoClust [10], which captures that "one of the most commonly-used approaches by non-expert users is to try all possible clustering algorithms with their default hyperparameters and then select the model with the best performance." Following this, we run the four River fixed algorithms (CluStream, DenStream, DBSTREAM, StreamKMeans) with their default settings and report the best-performing model per dataset.

TABLE II
SPECIFICATION OF OUR EVALUATION PLATFORM.

Component	Description
Processor	2× AMD EPYC 7702 64-Core @ 2.0 GHz (128 cores)
L3 Cache	512 MiB
Memory	1 TiB DDR4 RAM
OS	Ubuntu 24.04.4 LTS
Kernel	Linux 6.8.0-101-generic
Runtime	Python 3.12.3, River [21]
Compiler	GCC 13.3.0

TABLE III
SEARCH SPACE CONFIGURATION.

Algorithm	Hyperparameter	Type	Range/Options
CluStream	n_macro_clusters	integer	[2, 15]
	max_micro_clusters	integer	[20, 100]
	time_window	integer	[250, 1500]
	micro_cluster_r_factor	numeric	[1.0, 3.0]
DenStream	decaying_factor	numeric	[0.01, 0.2]
	beta	numeric	[0.2, 0.8]
	mu	numeric	[2.0, 8.0]
	epsilon	numeric	[0.01, 1.0]
DBSTREAM	clustering_threshold	numeric	[0.1, 2.0]
	fading_factor	numeric	[0.001, 0.05]
	intersection_factor	numeric	[0.1, 0.8]
	minimum_weight	numeric	[0.5, 5.0]
StreamKMeans	chunk_size	integer	[100, 500]
	n_clusters	integer	[2, 20]
	halflife	numeric	[0.05, 0.8]

D. Implementation and Configuration

Unless otherwise stated, AutoDSC uses its default configuration of population size $K = 10$ and window size $w = 1000$, with the exploration rate adapting automatically from an initial value of 0.5 (Section VI-B analyzes sensitivity to K and w). The AutoML baselines search over the same algorithm portfolio and hyperparameter ranges specified in Table III, where the window-dependent ranges (`time_window` and `chunk_size`) are listed for the default window size $w = 1000$; the single fixed algorithms use their library defaults. To ensure a fair comparison, all AutoML baselines receive the same window size and per-window evaluation budget as AutoDSC, so differences reflect the search and selection strategy rather than the compute allotment; other baseline-specific parameters remain as reported in their original papers.

All experiments were conducted on the server detailed in Table II, using parallelization across cores for efficiency. All clustering algorithms were implemented with the River online-learning library [21].

VI. Results Analysis

Table IV presents a comprehensive comparison of AutoDSC against five baselines across ten diverse real-world and synthetic datasets, evaluating clustering quality with both ARI and CMM. Following OnlineAutoClust's *default-baseline* [10], we summarize the single fixed algorithms by trying all four River algorithms at their default hyperparameters and reporting the best-performing model per dataset (denoted Default(Best)). AutoDSC instead selects and tunes configurations automatically.

AutoDSC achieves the highest overall mean ARI of **0.622**, ahead of every baseline, surpassing the strongest AutoML competitor ConfStream (0.572) by 5.0 points, OnlineAutoClust (0.558) by 6.4 points, RandomSearch (0.534) by 8.8 points, and the Default(Best) baseline (0.517) by 10.5 points, and it attains the best mean rank on both ARI and CMM. AutoDSC ranks first on ARI for 7 of the 10 datasets, with the largest margins on the drifting synthetic streams (e.g. Fast-RBF, Gradual, Expanding all ≥ 0.99, where even the best default reaches only 0.76–0.82) and on the harder real stream PenDigits (0.510 vs. 0.412 for the best default). It is outranked on Shuttle and Vehicle, where a single default algorithm is already near-optimal, and on Merge-Split, where OACE leads narrowly (0.985 vs. 0.963). Figure 4 shows the prequential trajectories, where AutoDSC stays at or above all baselines throughout the stream on both a real (PenDigits) and a synthetic (Expanding) workload, reflecting rapid adaptation as the stream evolves.

Figure 1 isolates AutoDSC's behaviour under drift on two synthetic streams. On *Merge-Split* (a), which contains a discrete merge-then-split event mid-stream (shaded), every adaptive method incurs a temporary drop in ARI and recovers within roughly fifteen windows; AutoDSC sustains the highest stable quality (ARI $\approx$ 1.0) on either side of the drift and recovers together with the strongest baselines, consistent with its adaptive-exploration design. On *Expanding* (b), where within-cluster variance grows continuously throughout the stream (gradient), AutoDSC reaches near-perfect ARI almost immediately and holds it, whereas ConfStream converges more slowly and OACE and MOStream plateau visibly below it. AutoDSC thus both recovers from abrupt drift and sustains quality under gradual drift through its adaptive exploration alone.

To assess these performance gains statistically, the Critical Difference (CD) diagrams in Figure 5 rank all methods using the Friedman test with the Nemenyi post-hoc procedure ($\alpha = 0.05$, CD$= 2.38$). AutoDSC attains the best (lowest) mean rank on both ARI (1.40) and CMM (2.20), ahead of every baseline. On ARI, AutoDSC is significantly better than RandomSearch, MOStream, and the Default(Best) baseline; its advantage over the strongest AutoML competitors (ConfStream and OACE, both at rank 3.60) falls just inside the critical difference, so with only ten datasets these are not separated by a statistically significant margin. AutoDSC nonetheless ranks first on both metrics.

Beyond clustering quality, AutoDSC remains practical in throughput and memory. Among the AutoML methods it is the fastest, processing **2.8$\times$ more instances/s** than ConfStream and **2.0$\times$ more** than OACE, while using less memory than both ($\approx$25 MB vs. 27 and 53 MB). The Default(Best) baseline is naturally faster as it performs no search, and RandomSearch trades quality for speed; AutoDSC instead occupies the favorable region of the throughput–memory plane (Figure 6), pairing the highest clustering quality with a low memory footprint. OACE shows the worst AutoML footprint (53 MB) and MOStream the highest overall (166 MB).

To quantify the contribution of each component, we ran a leave-one-out ablation over all real-world datasets ($\times 10$ seeds), disabling one mechanism at a time, namely recency-weighted elite ranking, the adaptive exploration rate, the multi-objective (3-CVI) score, and diversity-preserving pruning. (Replacing the guided PGSSO search by random configuration generation is already covered by the RandomSearch baseline.) Figure 7 reports each variant's score relative to the full model. On these comparatively stable real-world streams the effects are small and metric-dependent: removing recency weighting causes the largest ARI drop ($\sim$1.7%), while removing the multi-objective CVI score lowers CMM by $\sim$1.2% but barely affects ARI, indicating that the two components matter on complementary metrics; diversity pruning gives a small consistent gain, and the adaptive exploration rate has a negligible standalone effect here, consistent with its role being to accelerate recovery under the stronger drift of the synthetic streams (Figure 1). The full configuration is the most balanced across both metrics, supporting the integrated design.

Finally, AutoDSC adaptively reshapes its population of candidate algorithms as the stream evolves. Figure 8 shows the algorithm composition of the population across stream windows on the Insects dataset, chosen as a representative real-world stream with pronounced concept drift, where each cell is the base algorithm occupying a population slot at that window. AutoDSC continuously maintains a diverse mix and shifts the composition between CluStream, DenStream, DBSTREAM and StreamKMeans as the active concept changes, rather than committing to a single algorithm.

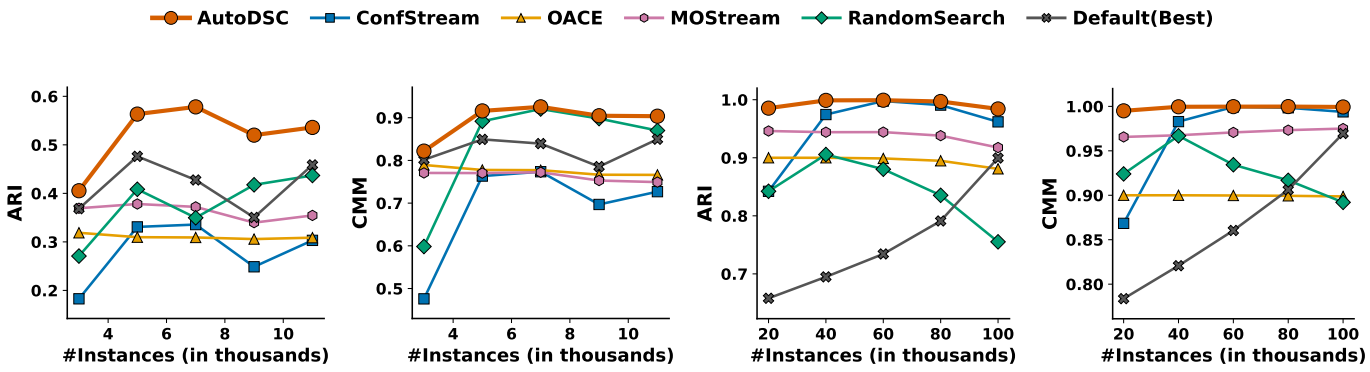
A. Supplementary Analyses

Label-free selection is sound. AutoDSC selects the incumbent each window with a label-free internal criterion, so its value hinges on that criterion tracking the unobserved external quality. On a real (PenDigits) and a drifting synthetic (Sudden) stream we bucket each window into terciles of the incumbent's internal multi-CVI score and measure the mean external ARI per bucket (Table V). On PenDigits, ARI rises monotonically with the internal score, from 0.33 in the lowest tercile to 0.48 in the highest (Spearman $\rho = 0.43$); on Sudden nearly every window already attains ARI ≈ 1.0, leaving little headroom to separate ($\rho = 0.17$). The label-free score is thus a sound, if imperfect, basis for selection, consistent with the limitation noted in Section VII. More strikingly, although AutoDSC never observes the number of clusters, the incumbent's mean

TABLE IV

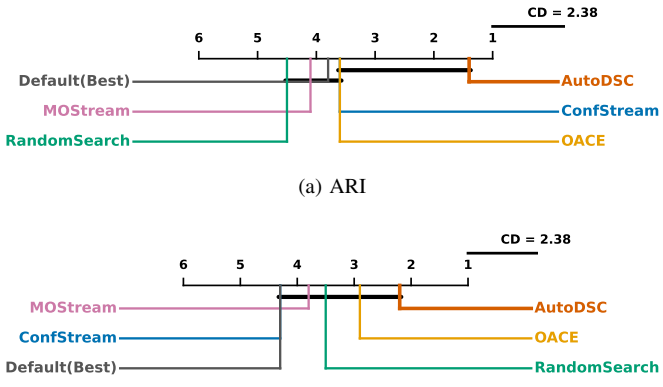
CLUSTERING QUALITY (ARI AND CMM, ↑ HIGHER IS BETTER) ACROSS DATASETS OVER 10 SEEDS. PER-DATASET CELLS REPORT MEAN±STD OVER SEEDS, WHILE THE MEAN SCORE ROW REPORTS THE MEAN ACROSS DATASETS. BEST PER METRIC PER ROW IN **BOLD**, SECOND BEST UNDERLINED.

Dataset	AutoML						Rule-based		Baselines			
	AutoDSC		ConfStream		OACE		MOStream		RandomSearch		Default(Best)	
	ARI	CMM	ARI	CMM	ARI	CMM	ARI	CMM	ARI	CMM	ARI	CMM
Insects	**.117±.003**	.905±.024	.016±.004	.827±.019	.048±.035	**.969±.028**	.090±.045	.964±.006	.054±.007	.880±.019	.100±.002	.947±.001
KDD99	**.037±.000**	.942±.015	.037±.001	.910±.013	.036±.003	.626±.133	.018±.014	.780±.174	.033±.001	.935±.017	.037±.000	**.990±.000**
PenDigits	**.510±.012**	**.888±.014**	.271±.029	.668±.025	.311±.120	.777±.107	.363±.010	.764±.016	.367±.035	.814±.016	.412±.000	.823±.000
Shuttle	.363±.011	.900±.006	.341±.002	.887±.000	.146±.083	**.974±.034**	.002±.001	.380±.054	.270±.014	.903±.018	**.409±.000**	.886±.000
Vehicle	.320±.002	.975±.005	.303±.001	.967±.000	.312±.003	.976±.000	.335±.017	**.990±.001**	.172±.017	.977±.001	**.335±.003**	.986±.000
Expanding	**.993±.001**	**.999±.001**	.953±.028	.969±.020	.895±.298	.900±.300	.938±.111	.970±.058	.844±.017	.927±.011	.756±.103	.868±.055
Gradual	**.998±.001**	1.000±.000	.956±.042	.964±.032	.995±.014	**1.000±.000**	.978±.066	.984±.047	.921±.014	.978±.004	.759±.130	.838±.094
Merge-Split	.963±.004	.988±.001	.952±.009	.979±.007	**.985±.001**	**.999±.000**	.932±.102	.968±.062	.877±.012	.947±.009	.895±.054	.945±.034
Sudden	**.921±.001**	**.956±.001**	.904±.013	.939±.010	.855±.003	.884±.000	.774±.090	.841±.067	.878±.006	.947±.002	.646±.160	.757±.116
Fast-RBF	**.999±.001**	**1.000±.000**	.988±.001	.989±.000	.995±.008	.999±.002	.911±.109	.938±.076	.924±.009	.963±.005	.818±.096	.896±.056
Mean Score	**.622**	**.955**	.572	.910	.558	.910	.534	.858	.534	.927	.517	.894
Mean Rank	**1.40**	**2.20**	3.60	4.30	3.60	2.90	4.10	3.80	4.50	3.50	3.80	4.30



(a) PenDigits, ARI (b) PenDigits, CMM (c) Expanding, ARI (d) Expanding, CMM

Fig. 4. Prequential clustering quality across stream stages for all methods on a real (PenDigits) and a synthetic drifting (Expanding) stream, with AutoDSC drawn thick.



(a) ARI

(b) CMM

Fig. 5. Critical Difference diagrams (Nemenyi, $\alpha = 0.05$) for ARI and CMM; methods connected by a bar are not significantly different.

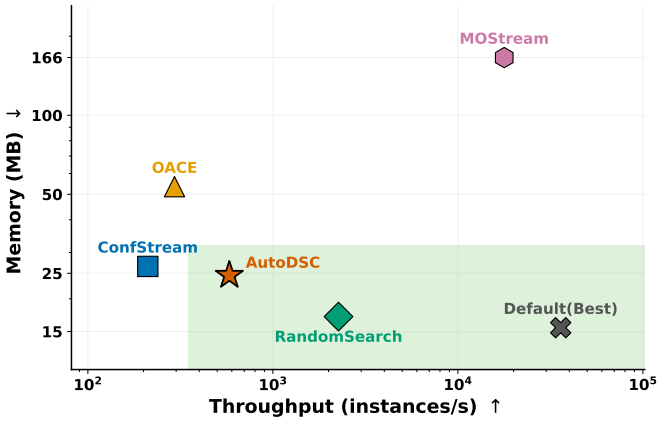


Fig. 6. Throughput vs. memory trade-off (mean over all datasets, log–log axes); the shaded region marks the favorable high-throughput, low-memory area, where AutoDSC (⋆) sits.

per-window cluster count matches the ground truth almost exactly (10.2 vs. 10 on PenDigits, 5.0 vs. 5 on Sudden), addressing the long-standing problem of choosing k online.

The search overhead is negligible. Decomposing AutoDSC's per-window wall-clock into base-clusterer training/prediction, multi-CVI scoring, and the PGSSO search itself (Table V) reveals three well-separated tiers: training the K population members dominates ($\approx$ 1.0–1.3 s/window), multi-CVI scoring is an order of magnitude smaller ($\approx$ 0.1 s), and the AutoML search machinery is three orders smaller still ($\approx$ 1 ms, under 0.1% of the total). The price of automation is thus dwarfed by the clustering it automates, explaining AutoDSC's competitive throughput against the offline optimization of OnlineAutoClust.

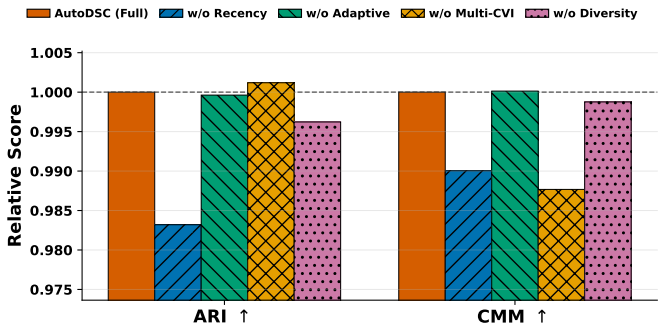


Fig. 7. Leave-one-out ablation on real-world streams, reporting ARI and CMM of each variant relative to the full AutoDSC (dashed line = 1.0).

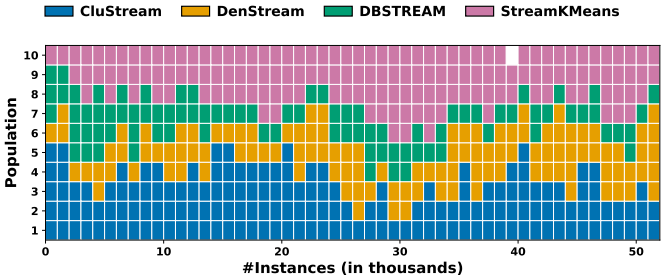


Fig. 8. Evolution of the AutoDSC population composition over the Insects stream, where colors denote the selected base algorithm per population slot (rows) and stream stage (columns).

Quality is stable across dimensionality. On a controlled five-cluster blob stream with feature dimensionality growing from 5 to 80 (Table VI), clustering quality stays high and even improves slightly (ARI $0.91 \rightarrow 0.97$) while throughput degrades only gracefully ($620 \rightarrow 376$ instances/s), with no curse-of-dimensionality collapse, indicating that AutoDSC's search remains effective as dimensionality increases.

B. Sensitivity Analysis

We analyze AutoDSC's sensitivity to its two free hyperparameters, window size w and population size K, averaged over all datasets ($\times 10$ seeds), jointly reporting clustering quality (ARI) and throughput (Figure 9). Window size shows a clear quality optimum at $w = 1000$, where smaller windows give insufficient data for reliable per-window CVI evaluation, while larger windows delay adaptation to drift and reduce throughput. Population size exhibits a quality plateau, where ARI rises steeply from $K = 5$ and saturates by $K \approx 15$, while throughput falls monotonically as K grows. The knee of this trade-off is at $K = 10$, which attains near-peak quality at substantially higher throughput, justifying it as the default. Overall, AutoDSC is stable within reasonable ranges and degrades gracefully outside them, with the default configuration ($w = 1000$, $K = 10$) sitting at the quality/efficiency sweet spot of both parameters.

TABLE V
SUPPLEMENTARY ANALYSES ON A REAL (PENDIGITS) AND A DRIFTING SYNTHETIC (SUDDEN) STREAM (MEAN OVER 10 SEEDS). AUTODSC NEVER OBSERVES LABELS OR THE CLUSTER COUNT k DURING THE STREAM.

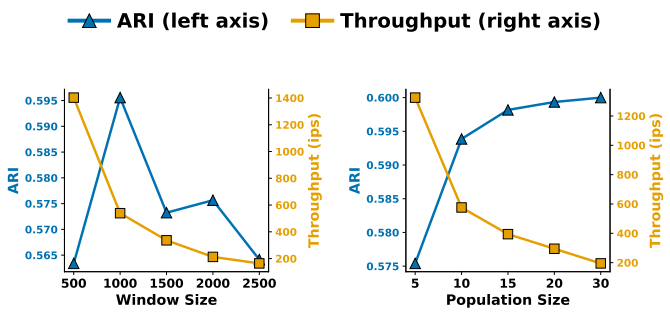
Measure	**PenDigits**	**Sudden**
Label-free selection validity		
Spearman ρ(internal score, ARI)	0.43	0.17
Predicted clusters (mean)	10.2	5.0
True cluster count k	10	5
Per-window runtime (ms)		
Train + predict	1039	1324
Multi-CVI scoring	122	94
PGSSO search	0.96	0.89
PGSSO search share (%)	0.08	0.06

TABLE VI
DIMENSIONALITY SCALABILITY OF AUTODSC ON A CONTROLLED FIVE-CLUSTER BLOB STREAM (MEAN OVER 10 SEEDS): QUALITY IS STABLE WHILE THROUGHPUT DEGRADES ONLY GRACEFULLY AS THE FEATURE COUNT GROWS.

Dimensionality	5	10	20	40	80
ARI ↑	0.910	0.944	0.958	0.963	0.974
Throughput (ips) ↑	620	686	601	509	376

VII. DISCUSSION AND LIMITATIONS

AutoDSC represents a significant advancement in AutoML for streaming environments through several key innovations. Our multi-objective evaluation strategy, combining three normalized clustering validation indices, provides robust performance assessment compared to single-metric approaches used in ConfStream (Silhouette Width only) and OnlineAutoClust's ensemble voting. This prevents optimization convergence to configurations that excel in one aspect while failing catastrophically in others [23], [24]. The PGSSO outperforms existing methods by dynamically adjusting exploration rates based on performance trends, enabling rapid parameter space exploration after concept drift while efficiently exploiting successful configurations during stable periods, contrasting with ConfStream's fixed ensemble strategy and OnlineAutoClust's computationally intensive offline Bayesian optimization [39]. While our current evaluation focuses on tabular data streams with numerical and categorical features, extensions to high-dimensional sparse text streams, multi-modal event streams, or image-based data would require domain-specific clustering algorithms beyond this scope. We emphasize that AutoDSC's clearest advantages are its full autonomy, requiring no manual tuning or labels, and its efficiency, together with statistically significant gains over untuned single algorithms and random search. Against the strongest AutoML competitors its clustering quality is on par or better but, with only ten datasets, not separated by a statistically significant margin (Figure 5), so we frame its value as delivering competitive quality fully automatically and at lower cost rather than as a uniform quality win. These contributions collectively bridge the gap between traditional AutoML approaches and the dynamic requirements of deploying real-time streaming applications in IoT, sensor networks, and analytics platforms, where manual configuration



(a) Window size (b) Population size

Fig. 9. Sensitivity of AutoDSC to window size and population size, averaged over all datasets.

is impractical [12].

Several limitations bound the scope of our claims. First, with only ten datasets the quality advantage of AutoDSC over the strongest AutoML competitors (ConfStream and OACE) is not statistically significant, although AutoDSC attains the best mean rank and significantly outperforms untuned single algorithms and random search. Second, our evaluation targets tabular streams with numerical and categorical features, so extension to sparse text, multi-modal, or image streams would require domain-specific base clusterers. Third, AutoDSC still exposes two free hyperparameters, the window size and the population size, which require sensible defaults, although it remains stable across a broad range of both. Finally, configuration selection relies on internal validity indices computed without labels, so online clustering quality is estimated rather than observed, and a misleading internal score can temporarily mislead selection between ground-truth evaluations.

VIII. Conclusion and Future Work

We presented AutoDSC, an AutoML framework that solves the combined algorithm selection and hyperparameter optimization problem for data stream clustering in a single pass. AutoDSC proposes configurations with a population-guided search over a recency-weighted elite archive and chooses among them using a label-free multi-objective criterion, adapting implicitly as the stream drifts. Across ten diverse data streams, AutoDSC attains the best mean rank on both ARI and CMM and the highest mean ARI of all methods, a statistically significant improvement over untuned single algorithms, while being the fastest and most memory-efficient of the AutoML stream clusterers. Future research directions include extending AutoDSC to semi-supervised settings, multi-task clustering, and advanced multi-objective optimization techniques that balance accuracy, efficiency, and robustness in dynamic settings.

References

[1] F. Hutter, L. Kotthoff, and J. Vanschoren, *Automated machine learning: methods, systems, challenges*. Springer Nature, 2019.

[2] Y. Poulakis, C. Doulkeridis, and D. Kyriazis, "A survey on automl methods and systems for clustering," *ACM Transactions on Knowledge Discovery from Data*, vol. 18, no. 5, pp. 1–30, 2024.

[3] A. Bifet, R. Gavaldà, G. Holmes, and B. Pfahringer, *Machine Learning for Data Streams with Practical Examples in MOA*. MIT Press, 2018. [Online]. Available: https://moa.cms.waikato.ac.nz/book/

[4] L. Yang and A. Shami, "Iot data analytics in dynamic environments: From an automated machine learning perspective," *Engineering Applications of Artificial Intelligence*, vol. 116, p. 105366, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0952197622003803

[5] B. Celik, P. Singh, and J. Vanschoren, "Online automl: An adaptive automl framework for online learning," *Machine Learning*, vol. 112, no. 6, pp. 1897–1921, 2023.

[6] N. Verma, A. Bifet, B. Pfahringer, and M. Bahri, "Asml: A scalable and efficient automl solution for data streams," in *Proceedings of the Third International Conference on Automated Machine Learning*, ser. Proceedings of Machine Learning Research, K. Eggensperger, R. Garnett, J. Vanschoren, M. Lindauer, and J. R. Gardner, Eds., vol. 256. PMLR, 09–12 Sep 2024, pp. 11/1–26. [Online]. Available: https://proceedings.mlr.press/v256/verma24a.html

[7] Q. Wu, C. Wang, J. Langford, P. Mineiro, and M. Rossi, "Chacha for online automl," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 18–24 Jul 2021, pp. 11 263–11 273. [Online]. Available: https://proceedings.mlr.press/v139/wu21d.html

[8] N. Verma, A. Bifet, B. Pfahringer, and M. Bahri, "Auto-reg: A dynamic automl framework for streaming regression," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 2025, pp. 245–256.

[9] M. Carnein, H. Trautmann, A. Bifet, and B. Pfahringer, "confstream: Automated algorithm selection and configuration of stream clustering algorithms," in *International conference on learning and intelligent optimization*. Springer, 2020, pp. 80–95.

[10] R. El Shawi and D. Rozgonjuk, "Onlineautoclust: A framework for online automated clustering," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, 2023, pp. 3870–3874.

[11] Z. Wang, X. Wang, and S. Zhang, "MOStream: A modular and self-optimizing data stream clustering algorithm," in *2024 IEEE International Conference on Data Mining (ICDM)*, 2024, pp. 500–509.

[12] M. Bahri, A. Bifet, J. Gama, H. M. Gomes, and S. Maniu, "Data stream analysis: Foundations, major tasks and tools," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 11, no. 3, p. e1405, 2021.

[13] L. Hubert and P. Arabie, "Comparing partitions," *Journal of Classification*, vol. 2, no. 1, pp. 193–218, 1985.

[14] H. Kremer, P. Kranen, T. Jansen, T. Seidl, A. Bifet, G. Holmes, and B. Pfahringer, "An effective evaluation measure for clustering on evolving data streams," in *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '11. New York, NY, USA: Association for Computing Machinery, 2011, pp. 868–876. [Online]. Available: https://doi.org/10.1145/2020408.2020555

[15] C. C. Aggarwal, "Data streams: models and algorithms," *Springer Science & Business Media*, 2013.

[16] T. Zhang, R. Ramakrishnan, and M. Livny, "BIRCH: An efficient data clustering method for very large databases," *SIGMOD Rec.*, vol. 25, no. 2, pp. 103–114, 1996.

[17] C. C. Aggarwal, S. Y. Philip, J. Han, and J. Wang, "A framework for clustering evolving data streams," in *Proceedings 2003 VLDB conference*. Elsevier, 2003, pp. 81–92.

[18] F. Cao, M. Ester, W. Qian, and A. Zhou, "Density-based clustering over an evolving data stream with noise," in *SIAM International Conference on Data Mining (SDM)*, 2006. [Online]. Available: https://epubs.siam.org/doi/abs/10.1137/1.9781611972740.9

[19] M. Hahsler and M. Bolaños, "Clustering data streams based on shared density between micro-clusters," *IEEE Transactions on Knowledge and Data Engineering*, 2016. [Online]. Available: https://ieeexplore.ieee.org/document/7425260

[20] D. Sculley, "Web-scale k-means clustering," in *WWW 2010, Raleigh, North Carolina, USA*, 2010. [Online]. Available: http://code.google.com/p/sofia-ml

[21] J. Montiel, M. Halford, S. M. Mastelini, G. Bolmier, R. Sourty, R. Vaysse, A. Zouitine, H. M. Gomes, J. Read, T. Abdessalem *et al.*,

"River: machine learning for streaming data in python," *Journal of Machine Learning Research*, vol. 22, no. 110, pp. 1–8, 2021.

[22] L. O'Callaghan, N. Mishra, A. Meyerson, S. Guha, and R. Motwani, "Streaming-data algorithms for high-quality clustering," in *Proceedings of the 18th International Conference on Data Engineering (ICDE'02)*, 2002. [Online]. Available: http://www-db.stanford.edu/~loc/icde02.ps

[23] O. Arbelaitz, I. Gurrutxaga, J. Muguerza, J. M. Pérez, and I. Perona, "An extensive comparative study of cluster validity indices," *Pattern recognition*, vol. 46, no. 1, pp. 243–256, 2013.

[24] Y. Liu, Z. Li, H. Xiong, X. Gao, and J. Wu, "Understanding of internal clustering validation measures," in *2010 IEEE International Conference on Data Mining*, 2010, pp. 911–916.

[25] D. Tschechlov, M. Fritz, and H. Schwarz, "Automl4clust: Efficient automl for clustering analyses," in *Proceedings of the 24th International Conference on Extending Database Technology (EDBT)*. OpenProceedings.org, 2021, pp. 343–348.

[26] Y. Liu, S. Li, and W. Tian, "Autocluster: Meta-learning based ensemble method for automated unsupervised clustering," in *Advances in Knowledge Discovery and Data Mining (PAKDD 2021)*. Springer, 2021, pp. 246–258.

[27] R. ElShawi, H. Lekunze, and S. Sakr, "csmartml: A meta learning-based framework for automated selection and hyperparameter tuning for clustering," in *2021 IEEE International Conference on Big Data (Big Data)*. IEEE, 2021, pp. 1119–1126.

[28] D. Treder-Tschechlov, M. Fritz, H. Schwarz, and B. Mitschang, "Ml2dac: Meta-learning to democratize automl for clustering analysis," *Proc. ACM Manag. Data*, vol. 1, no. 2, Jun. 2023. [Online]. Available: https://doi.org/10.1145/3589289

[29] R. ElShawi and S. Sakr, "Tpe-autoclust: A tree-based pipline ensemble framework for automated clustering," in *2022 IEEE International Conference on Data Mining Workshops (ICDMW)*, 2022, pp. 1144–1153.

[30] M. C. da Silva, G. M. Tavares, E. Medvet, and S. B. Junior, "Problem-oriented automl in clustering," *arXiv preprint arXiv:2409.16218*, 2024, arXiv:2409.16218.

[31] X. Wang, Z. Wang, Z. Wu, S. Zhang, X. Shi, and L. Lu, "Data stream clustering: An in-depth empirical study," *Proc. ACM Manag. Data*, vol. 1, no. 2, 2023.

[32] V. M. Souza, D. M. dos Reis, A. G. Maletzke, and G. E. Batista, "Challenges in benchmarking stream learning algorithms with real-world data," *Data Mining and Knowledge Discovery*, vol. 34, no. 6, pp. 1805–1858, 2020.

[33] D. Dua and C. Graff, "Uci machine learning repository," 2019. [Online]. Available: http://archive.ics.uci.edu/ml

[34] M. Duarte and Y. H. Hu, "Vehicle classification in distributed sensor networks," *Journal of Parallel and Distributed Computing*, vol. 64, no. 7, pp. 826–838, 2004.

[35] A. Bifet, G. Holmes, R. Kirkby, and B. Pfahringer, "Moa: Massive online analysis," *The Journal of Machine Learning Research*, vol. 11, pp. 1601–1604, 2010.

[36] W. M. Rand, "Objective criteria for the evaluation of clustering methods," *Journal of the American Statistical association*, vol. 66, no. 336, pp. 846–850, 1971.

[37] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine learning research*, vol. 7, no. Jan, pp. 1–30, 2006.

[38] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *The journal of machine learning research*, vol. 13, no. 1, pp. 281–305, 2012.

[39] F. Hinder, A. Artelt, and B. Hammer, "Towards non-parametric drift detection via dynamic adapting window independence drift detection (dawidd)," in *International Conference on Machine Learning*. PMLR, 2020, pp. 4249–4259.